
status: implemented date: 2026-05-18 implemented: 2026-05-18 methodology: [MADR] decision-makers: [Henrik Pettersen] tags: [pipeline, ci, napi, fail-loud, ADR-0082] related: [0082, 0133, 0162, 0186] audience: ai-executor

ADR-0189: Fail-loud detection for new NAPI crates outside `NAPI_PACKAGES` allowlist

Context and Problem Statement

ADR-0186's "Pipeline silently-drops new artifacts" audit (originally ADR-0162 follow-up #3) confirmed:

- `lib/napi-config.sh` has **8 hardcoded `NAPI_PACKAGES` entries**:

```
FORK_DIR_RUVECTOR:crates/ruvector-graph-node:npm/packages/graph-node
FORK_DIR_RUVECTOR:crates/ruvector-node:npm/packages/core
FORK_DIR_RUVECTOR:crates/ruvector-router-ffi:npm/packages/router
FORK_DIR_RUVECTOR:crates/ruvector-tiny-dancer-node:npm/packages/tiny-dancer
FORK_DIR_RUVECTOR:crates/sona:npm/packages/sona
FORK_DIR_RUVECTOR:examples/ruvLLM:npm/packages/ruvllm
FORK_DIR_RUVECTOR:crates/rvf/rvf-node:npm/packages/rvf-node
FORK_DIR_AGENTIC:packages/agentik-jujutsu:packages/agentik-jujutsu
```

- `forks/ruvector/crates/` ships **120+ crates** today (`agentik-robotics-*` , `neural-trader-*` , `ruvector-*-node` , `ruvector-*-wasm` , ...).
- **No fail-loud detection** exists for the case where a new NAPI crate appears upstream that isn't in `NAPI_PACKAGES` .

Today most non-listed crates are intentionally non-NAPI: Rust-only libs, WASM-only outputs, or examples / benches. But the audit can't tell *intent* from *omission* — a new

`ruvector-foo-node` could ship upstream tomorrow, get pulled in via the next upstream sync, and be silently absent from the `npm run release` build because nobody remembered to add it to `NAPI_PACKAGES`.

This is the exact class of silent-drop hazard ADR-0082 names: a missing artifact looks the same as an intentional skip, and only manifests as a runtime "module not found" failure on some downstream consumer weeks later.

Decision Drivers

- **ADR-0082 spirit** — fail loud, not silent. Silent omission of an NAPI crate that ships from upstream is the same failure mode ADR-0082 was designed to prevent.
- **Sync-wave cost** — every upstream sync adds crates. The cost of manual review grows. Automation catches what humans miss.
- **False-positive cost** — most crates are intentionally non-NAPI. A naive "every `ruvector-*-node` crate must be in `NAPI_PACKAGES`" check would flood with noise.
- **Where to enforce** — pre-flight (block `npm run release`), CI (block PR merge), advisory (warn but allow). Different ergonomics.
- **Cross-fork scope** — the same problem exists in `ruflo`, `agentic-flow`, `ruvector`, `ruv-FANN`, `agentdb` forks (though `ruvector` is the volume case).

Considered Options

1. **CI-only fail-loud check.** A new `scripts/check-napi-coverage.mjs` walks each `forks/*/crates/*/Cargo.toml` (and `forks/*/packages/*/Cargo.toml`, etc.), parses dependencies for `napi-derive` or `build.rs napi.build()`, and cross-checks against `NAPI_PACKAGES`. CI runs it as a gate on merges to `ruflo-patch:main`. Pre-flight skips it (so dev cycles stay fast).
2. **Pre-flight fail-loud check.** Same script, but invoked from `scripts/ruflo-publish.sh` before the build phase. Blocks `npm run release` entirely if an unlisted NAPI crate is detected.
3. **Advisory warn-only.** Same script, but emits a non-blocking warning. Easy to ignore; defeats ADR-0082.
4. **Per-crate intent declaration.** Every fork crate's `Cargo.toml` gains a `[package.metadata.ruflo]` section declaring `npm-publish = true | false`. The

check is then "does intent match NAPI_PACKAGES membership?" rather than "does the crate use napi-derive ?" — eliminates false positives but requires touching every fork crate.

Decision Outcome

Option 2 implemented — pre-flight gate, wired into `scripts/ruflo-publish.sh` .

Detector + tests + wiring all landed 2026-05-18; first GREEN run reports 19 NAPI crates, all listed in NAPI_PACKAGES .

What landed

- **`scripts/check-napi-coverage.mjs`** (~220 LOC, ESM, zero external deps). Recognizes both publish paths:
 - **Path 1 — NAPI_PACKAGES allowlist** (build-from-source via `lib/napi-config.sh`).
 - **Path 2 — republish via codemod rename** (crate has `package.json` + pre-built `.node` binaries in `npm/<platform>/` ; `copy-source.sh` + `codemod.mjs` rename `@ruvector/*` → `@sparkleideas/ruvector-*` ; publish ships them).
 - **No `package.json` → skip** (Rust-only workspace member; not an npm consumer).
- **`tests/unit/adr0189-napi-coverage.test.mjs`** — 8 unit tests covering: covered case, uncovered case, `workspace.dependencies` ignored, `target.cfg(...)` flagged, no-`package.json` skipped, Path 2 republish covered, Path 2 incomplete (`package.json` without pre-built binaries) flagged, and script-exists smoke check. All 8 pass (`node --test` 213ms). Full unit suite stays green: 3947 pass / 0 fail / 6 skipped.
- **Wired into `scripts/ruflo-publish.sh`** as the `napi-coverage` phase, slotted between `merge-detect` and `napi-rebuild` . Fails loud (`run_phase` exits the pipeline) before any version bump pollutes fork main.

Triage outcome — the 11 first-run gaps

Closed under user policy "always @sparkleideas" + the corrected detector:

- **3 republish-eligible** (`ruvector-attention-node` , `ruvector-gnn-node` , `ruvector-graph-transformer-node`) → recognized by Path 2; not gaps after the script fix. Already publishing as `@sparkleideas/ruvector-{attention,gnn,graph-transformer}` on Verdaccio.
- **2 added to NAPI_PACKAGES** (`ruvector-solver-node` , `agentic-robotics-node`) → now build from source via Path 1. Entries added to `lib/napi-config.sh` as `FORK_DIR_RUVECTOR:crates/<name>:crates/<name>` (single-binary pattern, same as `agentic-jujutsu`).
- **6 no-package.json crates** (`ruvector-attention` , `ruvector-gnn` , `ruvector-diskann-node` , `ruvector-mincut-node` , `examples/exo-ai-2025/crates/exo-node` , `agentic-flow/packages/agent-boostercrates/agent-boosternative`) → skipped by the script's "no package.json → Rust-only library" rule. Audited as legitimate workspace members consumed via path dependency by other crates; not standalone npm packages.

After fix: `node scripts/check-napi-coverage.mjs` reports OK: 19 NAPI crates, all listed in `NAPI_PACKAGES` , exit 0.

Background details preserved below

The original "Considered Options" + "Consequences" sections are kept for traceability of the alternatives.

- **Detector landed:** `scripts/check-napi-coverage.mjs` (178 lines, ESM, zero external deps) — parses `lib/napi-config.sh` `NAPI_PACKAGES` , walks every `Cargo.toml` under each fork in `config/upstream-branches.json` , flags crates that declare `napi-derive` outside `[workspace.dependencies]` , cross-checks by `forkVar + relCratePath` .
- **Unit test landed:** `tests/unit/adr0189-napi-coverage.test.mjs` — 5 tests covering: covered case, uncovered case, workspace-only ignored, `target.cfg(...)` dependencies flagged, script-exists smoke check. All 5 pass (`node --test` 131ms).
- **Wiring NOT landed:** the script was supposed to slot into `scripts/ruflo-publish.sh` pre-flight. First run surfaced 11 unlisted NAPI-consumer crates. Landing the wiring now would brick every `npm run release` invocation — the opposite of the intent.

Real gap surfaced (first invocation, 2026-05-18)

11 NAPI-consumer crates exist in our forks but aren't in `NAPI_PACKAGES` :

Has `package.json` and `-patch.N` versions — npm shippers (5):

- `forks/ruvector/crates/ruvector-attention-node` → `@ruvector/attention` (0.1.4-patch.122)
- `forks/ruvector/crates/ruvector-gnn-node` → `@ruvector/gnn` (0.1.25-patch.117)
- `forks/ruvector/crates/ruvector-graph-transformer-node` → `@ruvector/graph-transformer` (2.0.4-patch.117)
- `forks/ruvector/crates/ruvector-solver-node` → `@ruvector/solver` (0.1.0-patch.95)
- `forks/ruvector/crates/agentic-robotics-node` → `agentic-robotics` (0.1.3)

Cross-reference: 3 of these (`-attention-node` , `-gnn-node` , `-graph-transformer-node`) ARE referenced by `scripts/install-native-deps.sh` (the runtime install path that pulls prebuilt NAPI binaries from npm). They are not in `NAPI_PACKAGES` because we don't currently build+publish them as `@sparkleideas/*` mirrors — consumers fall back to upstream `@ruvector/*` packages via `install-native-deps.sh` . The other 2 (`-solver-node` , `agentic-robotics-node`) aren't even in `install-native-deps.sh` .

No `package.json` — not currently npm packages (5):

- `forks/ruvector/crates/ruvector-attention` (lib crate; `optional = true`)
- `forks/ruvector/crates/ruvector-gnn` (lib crate; `optional = true` under `target.cfg(linux)`)
- `forks/ruvector/crates/ruvector-diskann-node` (no `package.json` despite `-node` suffix)
- `forks/ruvector/crates/ruvector-mincut-node` (no `package.json` despite `-node` suffix)
- `forks/ruvector/examples/exo-ai-2025/crates/exo-node` (under `examples/`)

Plus 1 in `agentic-flow`:

- `forks/agentic-flow/packages/agent-booster/crates/agent-booster-native`

Follow-up — open

The wiring deferral creates explicit triage work. Three choices for each surfaced crate (multi-hour exercise to do per-crate):

1. **Add to `NAPI_PACKAGES`** — if we should be mirror-publishing it as `@sparkleideas/<name>` .
2. **Add to an explicit "intentionally-excluded" list** — a new file (e.g. `lib/napi-excludes.txt`) the detector reads as a SKIP layer. Documents intent; satisfies `feedback-no-fallbacks` (no silent exclusion).
3. **Add `[package.metadata.ruflo] napi-ship = false`** to each crate's `Cargo.toml` (Option 4 from the original ADR). High signal-to-noise but cross-fork churn.

Wiring lands once the 11 gaps are triaged. Until then, run the script manually: `node scripts/check-napi-coverage.mjs` (exits 1 on the current 11 misses, prints the list).

This ADR closes when the wiring lands and the script exits 0 by default on `main` . Option 1 lets devs iterate locally without the check; Option 2 catches the bug at every `npm run release` call but slows iteration.

Consequences

If Option 1 (CI-only):

- No `npm run release` slowdown.
- PR merges blocked until allowlist matches.
- Local `npm run release` can ship an inconsistent state if the dev doesn't push the PR.

If Option 2 (pre-flight):

- `npm run release` cycle adds a sub-second check (toml parse + glob walk).
- No local-vs-CI divergence: same gate everywhere.
- Slight friction for dev iteration; mitigated by `ACCEPTANCE_HEAVY=0` -style opt-out env var if needed.

If Option 3 (advisory):

- Cheap to implement; defeats the point.

If Option 4 (per-crate intent declaration):

- Highest signal-to-noise; explicitly documents intent.
- High upfront cost — every fork crate gets a `[package.metadata.rufflo]` section.
- Drift risk: future upstream crates land without the section, the check has to handle "missing-metadata" as a separate case.

This ADR closes when one of the options is implemented.